SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING

TO : MR. S YUTE

FROM : HOPE KUTENGULE AND MARRIAM HAJI

REG NO : BSC/ COM/NE/04/22 AND BSC/COM/NE/28/23

COURSE CODE : NET322

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION TO DEVELOPMENT

ASSIGNMENT TITLE : MOBILE APP DEVELOPMENT ASSIGNMENT 2

DUE DATE : 9th MAY 2026

SYSTEM ARCHITECTURE DIAGRAM

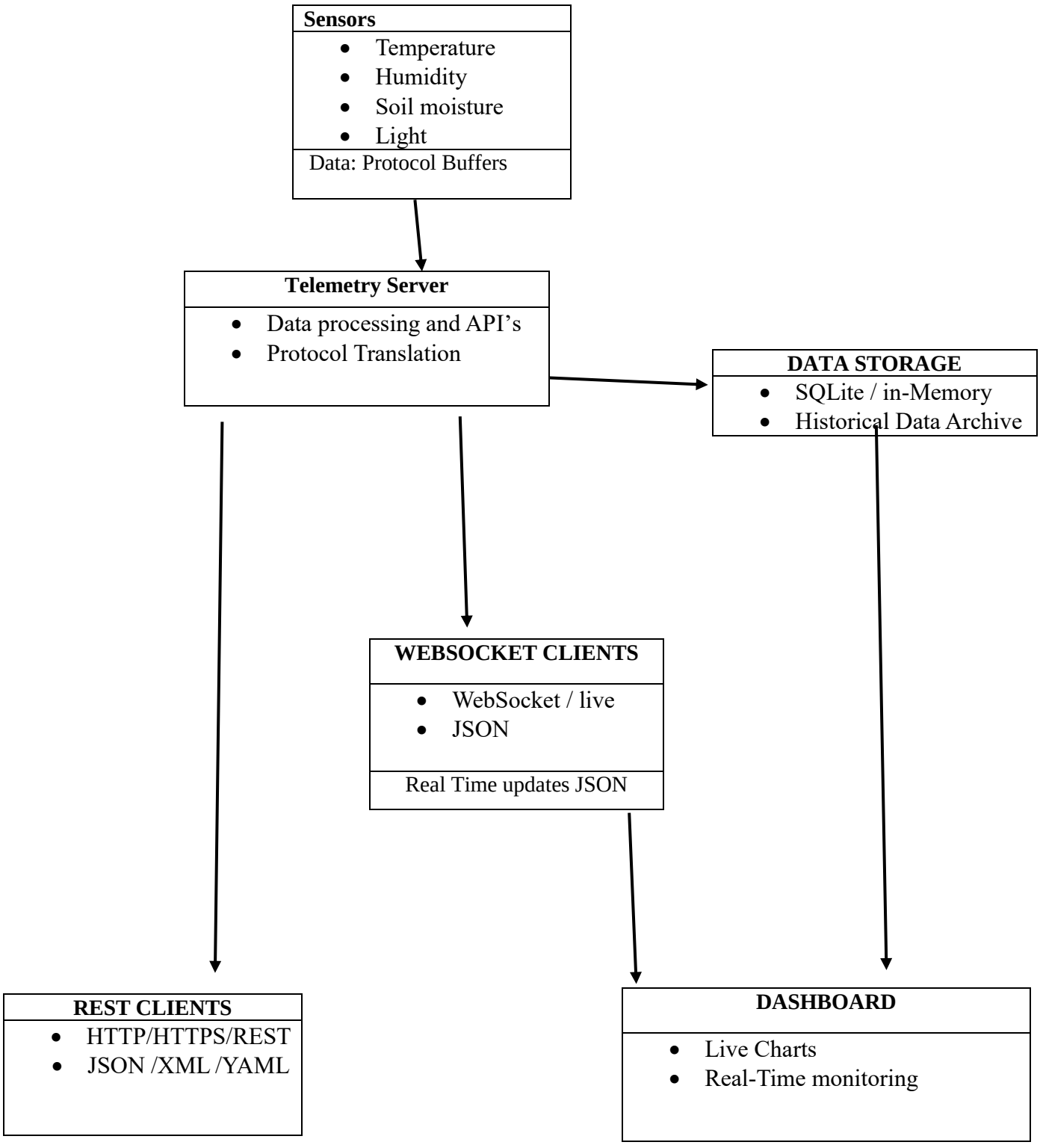
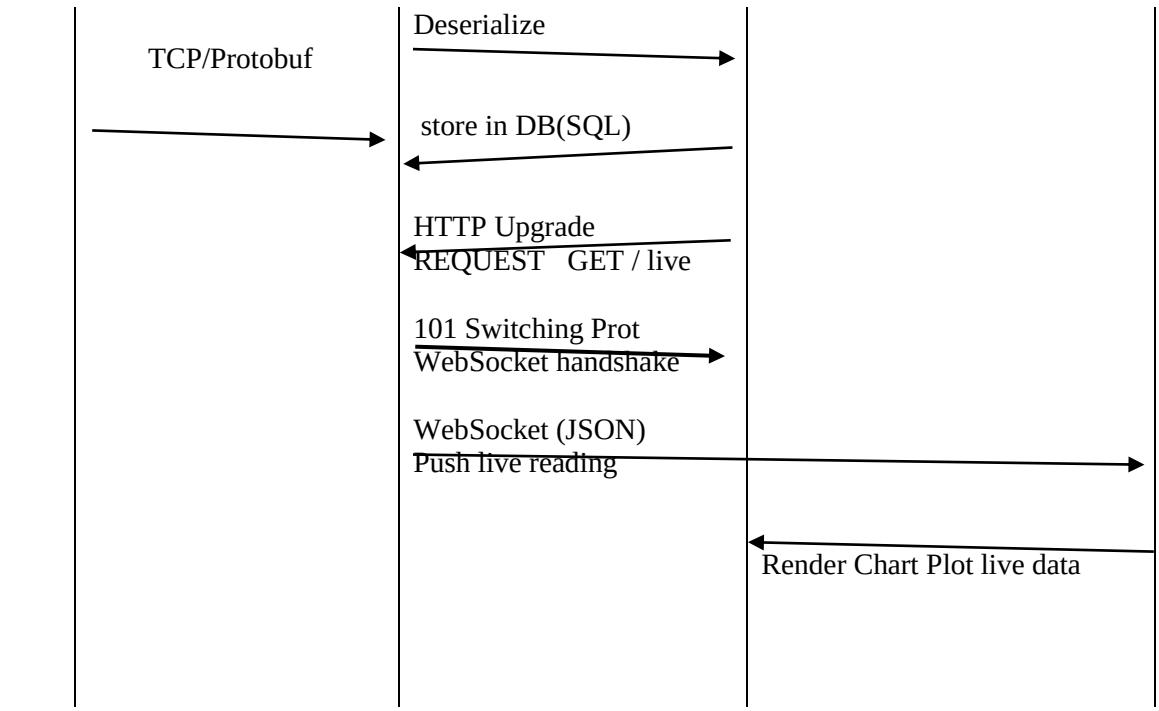


TABLE OF LINK OF PROTOCOLS AND SERIALIZATION FORMATS

Link	Protocol	Serialization
Sensor Simulator to Telemetry Server	TCP	Protocol Buffers
Telemetry Server to Storage	Internal DB	SQLite / In-memory
Telemetry Server to REST Clients	HTTP / REST	JSON / XML / YAML (via Accept header)
Telemetry Server to WebSocket Clients	WebSocket	JSON

SEQUENCE DIAGRAM



Data Model (Persistent Entities)

Sensor

id (TEXT, PK)

name (TEXT)

type (TEXT: TEMPERATURE, HUMIDITY, SOIL_MOISTURE, LIGHT)

location (TEXT, optional)

interval (INTEGER, reporting interval in seconds)

registered_at (INTEGER, epoch millis)

Reading

id (INTEGER, PK, AUTOINCREMENT)

sensor_id (TEXT, FK Sensor.id)

timestamp (INTEGER, epoch millis)

value (REAL, measurement value)

```
CREATE TABLE Sensor (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  type TEXT NOT NULL,
  location TEXT,
  interval INTEGER NOT NULL,
  registered_at INTEGER NOT NULL
);

CREATE TABLE Reading (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  sensor_id TEXT NOT NULL,
  timestamp INTEGER NOT NULL,
  value REAL NOT NULL,
  FOREIGN KEY (sensor_id) REFERENCES Sensor(id)
);
```

```
syntax = "proto3”;
package greenhouse;
        // Sensor entity definition
message Sensor {
  string id = 1;
  string name = 2;
  string type = 3;
  string location = 4;
  int32 interval = 5;
  int64 registered_at = 6;
}

// Reading entity definition
message Reading {
  string sensor_id = 1;
  int64 timestamp = 2;
  double value = 3;
}

// Envelope for sensor → server communication
message SensorReadingEnvelope {
  Sensor sensor_id= 1;
  Reading Reading_id = 2;
}

// REST API responses
message Sensor_list {
  repeated Sensor sensors = 1;
}
```

```
message Reading List {
  repeated Reading readings = 1;
}
```

Persistent Entities:

Sensor

Represents each physical or simulated sensor.

id (TEXT, PK)

name (TEXT)

type (TEXT)

location (TEXT, optional)

interval (INTEGER)

registered at (INTEGER)

ENTITY: Reading

Represents a single measurement from a sensor.

Reading _id (INTEGER, PK, AUTOINCREMENT)

sensor_id (TEXT, FK → Sensor.id)

timestamp (float)

temperature(float)

humidity(float)

Optional: Subscription

Tracks WebSocket clients subscribing to sensors.

client_id (TEXT)

sensor_id (ARRAY of TEXT)

SQLite Schema (DDL)

```
CREATE TABLE Sensor (
   Sensor_id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    type TEXT NOT NULL,
    location TEXT NOT NULL,
    interval INTEGER NOT NULL,
status TEXT NOT NULL DEFAULT ‘ONLINE’,
    registered_at INTEGER NOT NULL
);

CREATE TABLE Reading (
   Reading_id INTEGER PRIMARY KEY AUTOINCREMENT,
    sensor_id TEXT NOT NULL,
humidity REAL,
temperature REAL;
    timestamp DATETIME NOT NULL,
    value REAL NOT NULL,
    FOREIGN KEY(sensor_id)
   REFERENCES Sensor(sensor_id)
  ON DELETE CASCADE
);
```

Full proto File

```
proto
syntax = "proto3";

package greenhouse;

message Sensor {
  string sensor_id = 1;
```

```
  string name = 2;
  string type = 3;
  string location = 4;
  int64 registered_at = 5;
}

message Reading {
  string sensor_id =1;
  int64 timestamp =2;
  double value = 3;
}

// Envelope for sensor → server communication
message SensorReadingEnvelope {
  Sensor sensor_id = 1;
  Reading Reading_id = 2;
}

// REST API responses
message SensorList {
  repeated Sensor sensors = 1;
}

message ReadingList {
  repeated Reading readings = 1;
}
```

REST ENDPOINT SPECIFICATION

Sensor Data Ingestion

Field	**Description**
Path	/api/v1/sensors/data
method	POST
REQUEST BODY	{ "sensor_Id": "string", "timestamp": "ISO8601", "value": "number" }
Response Body	{ "status": "success", "record_Id": "string" }
Expected Status Codes	201 Created, 400 Bad Request, 500 Internal Server Error
Example	**Request:** POST /api/v1/sensors/data { "sensor_Id": "temp-001", "timestamp": "2026-05-09T21:18:00Z", "value": 23.5 } **Response:** { "status": "success", "record_Id": "abc123" }

Retrieve stored sensor Data

Field	**Description**
Path	/api/v1/sensors/{sensor_Id}/records
Method	GET
Request Body	None

Response Body	[{ "timestamp": "ISO8601", "value": "number" }]
Expected Status Codes	200 OK, 404 Not Found, 500 Internal Server Error
Example	**Request:** GET /api/v1/sensors/temp-001/records **Response:** [{ "timestamp": "2026-05-09T21:18:00Z", "value": 23.5 }]

Real-Time Updates via WebSocket

Field	**Description**
Path	/ws/sensors/live
Method	WebSocket
Request Body	{"subscribe": ["sensorId1", "sensorId2"]}
Response Body	Stream of JSON messages: { "sensor_Id": "string", "timestamp": "ISO8601", "value": "number" }
Expected Status Codes	101 Switching Protocols, 400 Bad Request
Example	Connection**:** ws://server/ws/sensors/live Message**:** {"subscribe": ["temp-001"]} Response Stream**:** {"sensor_Id": "temp-001", "timestamp": "2026-05-09T21:18:00Z", "value": 23.5}

Storage Synchronization

Field	Description
Path	/api/v1/storage/sync
Method	PUT
Request Body	{ "records": [{ "sensor_Id": "string", "timestamp": "ISO8601", "value": "number" }] }
Response Body	{ "status": "synced", "count": "integer" }
Expected Status Codes	200 OK, 400 Bad Request, 500 Internal Server Error
Example	Request**:** PUT /api/v1/storage/sync { "records": [{ "sensor_Id": "temp-001", "timestamp": "2026-05-09T21:18:00Z", "value": 23.5 }] } Response: { "status": "synced", "count": 1 }

YAML CONFIGURATION SCHEMA

```
Sensors:
    id: "temp-001”
   type: "temperature"
    location: "greenhouse-1"
    sampling_interval: 30
    protocol: "MQTT"
    encoding: "JSON"
    thresholds:
      min: 10.0
```

```
    max: 35.0
  connection:
    host: "iot-broker.local"
    port: 1883
    topic: "sensors/temperature/greenhouse-1"
  security:
    auth_token: "abc123securetoken"
    tls_enabled: true
    cert_path: "/etc/certs/temp-001.pem"
  metadata:
    manufacturer: "SensTech"
    model: "STX-200"
    calibration_date: "2026-04-15"

- id: "hum-002"
  type: "humidity"
  location: "greenhouse-1"
  sampling_interval: 45
  protocol: "CoAP"
  encoding: "Protocol Buffers"
  thresholds:
    min: 40.0
  max: 80.0
```

```
  connection:
  host: "iot-server.local"
  port: 5683
  topic: "/sensors/humidity/greenhouse-1"
  security:
  auth_token: "xyz789securetoken"
    tls_enabled: false
    cert_path: ""
   metadata:
    manufacturer: "EnviroSense"
    model: "ES-H100"
    calibration_date: "2026-03-20"
```

CONCURRENCY MODEL

In this concurrency model, the system is organized around a set of AsyncIO coroutines that each handle a distinct responsibility while communicating through asynchronous queues. Sensor reader tasks connect to devices via protocols like MQTT or CoAP, deserialize incoming payloads, and push them into a central sensor queue. A data processor coroutine consumes from this queue, validates and enriches the readings, and then distributes them into two downstream queues: one for database writing and another for broadcasting to clients. The database writer coroutine consumes records in batches and persists them to storage, while REST API handler coroutines are spawned by the web framework to serve synchronous request response interactions, awaiting database queries or queue submissions before returning JSON or XML responses. In parallel, a WebSocket broadcaster coroutine listens to the broadcast queue and pushes updates to all connected clients, using concurrent send operations to avoid blocking. Communication between these tasks is indirect, mediated by asyncio.Queue objects, which

naturally enforce backpressure when downstream consumers are slow. If a WebSocket client cannot keep up, the broadcaster's send operation may block; to prevent stalling the entire system, timeouts are applied and messages may be dropped for that client, possibly replaced with a notification that updates were missed. REST clients, being request-driven, simply experience longer latency or receive a timeout error if they are too slow. When queues fill up due to slow consumers, producers block until space is available, ensuring that the system either slows down gracefully or discards excess data rather than collapsing. This design keeps the system responsive, balances throughput, and makes explicit how slow consumers are handled without compromising the overall flow.

FAILURE HANDLING

In this system, failure handling is designed to ensure resilience and graceful degradation rather than outright collapse. When a **se**nsor disconnects, its reader coroutine detects the loss of connection and attempts automatic reconnection with exponential backoff. During downtime, the system may mark the sensor as "unavailable" and either buffer recent readings locally (if supported) or simply skip updates until the connection is restored. If the storage layer fails, the database writer coroutine encounters errors when attempting inserts; in this case, records are temporarily held in memory queues or written to a local fallback store (such as a file-based buffer) until the database becomes available again. If the outage persists and queues reach their maximum size, the system applies backpressure by blocking upstream producers, which slows ingestion and may eventually drop the oldest data to preserve memory. When the **server is overloaded**, AsyncIO's event loop begins to accumulate pending tasks, increasing latency. To mitigate this, the system can shed load by rejecting new REST requests with 503 Service Unavailable, throttling WebSocket broadcasts, or prioritizing critical sensor data over less urgent traffic. In all cases, the guiding principle is to fail fast and visibly logging errors, notifying operators, and applying backpressure or message dropping so that the system remains responsive and recovers cleanly once resources are restored.